

- Microservices repo is only for deployment “And for editing In case you need a backup for a rollback, etc (“record-keeping”)”
- “The point of the API gateway and the games is that each game is its own microservice. potentially in a real production environment, you could update a specific game without everything else going down or that how I see it at least it more or less just to decouple the games from the website” - Hensley
- Depreciated games in views/games
- Connect the web app to the API of the individual game microservice. Use an iframe in the web app to connect to the docker container
- Microservice HTTP get needs to be set up.
- API gateway needs to be set up to with HTTP get request to the docker container for the js
- Add endpoints to the controller inside the snake controller
- Add new or use existing snake controller to get the game from wwwroot

Current Data Flow and Structure

1. Web Application (BucStop)

- **Front-End:** The web app (BucStop) is built using HTML/CSS/JS for the user interface.
- **Backend (C# MVC):**
 - **Controllers:** Handle HTTP requests, interact with models, and return appropriate views.
 - **Models:** Represent the game metadata (title, author, description, etc.) from each microservice.
 - **Views:** Display game details and potentially the embedded game itself to the user.
 - **JSON API Calls:** The web app calls the **API Gateway** to retrieve game metadata (description, title, etc.).

2. API Gateway

- **Responsibility:** Acts as a bridge between the BucStop web app and the microservices.
- **Functionality:** Handles requests for game metadata. When the web app makes a GET request for game information, the API Gateway routes it to the correct microservice based on the requested game (Tetris, Pong, Snake).
- **Controller:** The API Gateway's controller fetches static game data from each microservice and sends it to the BucStop web app.

3. Microservices (Tetris, Snake, Pong)

- Each microservice currently stores static game data (e.g., title, description, author).
- These microservices expose an API endpoint (GET request) that provides this static information to the API Gateway.
- **Current Limitation:** These microservices only serve static data and do not contain the JavaScript game logic or facilitate game execution.

Challenges to Solve

1. Transferring Game Logic to Microservices

Goal: Instead of having static data only, each microservice should serve the actual game logic (JavaScript) and allow the game to run from the microservice rather than from the web app.

Steps to Achieve This:

- 1. Move Game Logic (JavaScript) to Microservices:**
 - Right now, the games (Tetris, Snake, Pong) are being executed on the BucStop web app using client-side JavaScript.
 - We need to transfer this JavaScript logic to each respective microservice. This means the Tetris microservice will serve the JavaScript code for Tetris, and the same for Pong and Snake.
- 2. Serve JavaScript from Microservices:**
 - Instead of having just API endpoints that return static data, each microservice should serve the full game logic
 - Modify each microservice to serve the entire game page. This might include:
 - **New Endpoints:** For example, the Tetris microservice could serve the game at /play, which returns the entire game logic
 - **Hosting Static Files:** Each microservice will need to host the static game files, similar to how a static website would serve HTML and JavaScript files.
- 3. API Gateway to Route Game Requests:**
 - The API Gateway will need to handle routing for game execution.
 - Example flow:
 - When the user clicks "Play Tetris," the BucStop web app will send a request to the API Gateway.
 - The API Gateway will forward this request to the Tetris microservice, which serves the game page with the JavaScript code to the browser.
 - The JavaScript game will then run in the user's browser, but the game files are served from the microservice.
- 4. Front-End Changes in BucStop:**
 - Instead of embedding or linking to local JavaScript files for the games, the front end of the BucStop app should fetch the game files from the respective microservice through the API Gateway.

- For example, when a user selects "Tetris" from the UI, the **BucStop** app will make a request to the API Gateway, and the gateway will return the game logic hosted by the **Tetris** microservice.
5. **Microservice Example Setup:**
- **Tetris Microservice:**
 - Provide endpoints such as **/api/gameinfo** for static data and **/play** for the game logic.
 - Similar setup can be done for **Snake** and **Pong** microservices.

2. Deployment Considerations

- Ensure each microservice can serve static files.
- Update API Gateway to route **/play** requests to the correct microservice and serve the game logic.
- Test the setup locally, and then deploy to AWS or another cloud provider using microservices architecture.

Next Steps

1. **Refactor each microservice** to serve static files for the game logic.
2. **Modify the API Gateway** to route game execution requests.
3. **Update the BucStop web app** to fetch and render games from the microservices.
4. **Deploy and test** the system to ensure smooth game loading and execution from the microservices.